RELATÓRIO TÉCNICO

Integração MongoDB + Redis com Python — Etapas I e II

Estudo de Caso: Monitoramento da Qualidade do Ar
Universidade Comunitária da Região de Chapecó — Unochapecó
Disciplina: Bancos de Dados e Novas Tecnologias
Professora: Mônica Tissiani De Toni Pereira
Integrantes: Ana Gheno, Beatriz Pavão, Cleidivan Stffens e Mel Biazi
Data: 29/10/2026

1. Introdução

O trabalho consiste no desenvolvimento de um protótipo de monitoramento da qualidade do ar, utilizando Python para integrar MongoDB e Redis. Sensores simulados registram PM2.5, PM10, CO_2, temperatura e umidade. O MongoDB mantém o histórico das medições; o Redis armazena os resumos estatísticos recentes em Hashes. Os resultados são apresentados no terminal, sem interface gráfica.

2. Ferramentas utilizadas

Windows 11; Ubuntu 26.04 LTS via WSL; Visual Studio Code; Python 3; MongoDB 8.0 executado em Docker; Redis; bibliotecas PyMongo e redis-py. O ChatGPT (OpenAI) foi utilizado como apoio à configuração do ambiente, elaboração e revisão de códigos e organização do relatório.

3. Arquitetura do protótipo

Fluxo: Python (cadastro e simulação) → MongoDB (coleções Sensores e Medicoes) → Python (consulta e agregação) → Redis (Hashes por sensor e cidade) → consulta no console.
[INSERIR AQUI A FIGURA DO DIAGRAMA DA ARQUITETURA]
O MongoDB é a fonte dos dados históricos. O processamento das estatísticas utiliza um pipeline com $match, $lookup, $unwind, $group, $avg, $max e $project. O Redis mantém os resultados da janela horária mais recente processada.

4. Desenvolvimento

4.1 Cadastro dos sensores — 01_cadastrar_sensores.py

O primeiro programa cadastra seis sensores na coleção Sensores do banco QualidadeAr: S001 e S002 em Chapecó, S003 e S004 em Xaxim, S005 e S006 em Concórdia. Cada sensor contém identificador, modelo, descrição, localização, estado de atividade e grandezas monitoradas.

4.2 Geração de medições — 02_gerar_medicoes.py

O segundo programa gera dados simulados durante 24 horas, em intervalos de cinco minutos. São 288 medições por sensor, totalizando 1.728 documentos na coleção Medicoes. Cada documento armazena sensor_id, timestamp, pm25, pm10, co2, temperatura e umidade. Foi criado índice sobre sensor_id e timestamp.

4.3 Consultas — 03_consultar_medicoes.py

O terceiro programa consulta as medições por sensor e intervalo temporal, usando filtros de data e ordenação cronológica. No teste, a consulta do sensor S001 de 20/10/2026, das 11h às 12h UTC, retornou 12 registros.

4.4 Estatísticas e Redis — 04_agregar_dados.py

O quarto programa agrupa as medições por sensor e cidade, calcula quantidade, médias de PM2.5, PM10, CO_2, temperatura e umidade, além dos máximos de PM2.5 e PM10. Os resultados são gravados no Redis em Hashes com chaves como qualidadear:sensor:S001 e qualidadear:cidade:SC:chapeco.

5. Resultados comprovados

O terminal confirmou o cadastro de seis sensores e a geração de 1.728 medições (288 para cada sensor). A consulta temporal do sensor S001 retornou 12 medições. A leitura do Hash de Chapecó no Redis, para 20/10/2026 das 23h às 00h UTC, exibiu os resultados abaixo:
Quantidade de medições: 24
PM2.5 médio: 25,21 µg/m³
PM10 médio: 50,61 µg/m³
CO_2 médio: 677,77 ppm
Temperatura média: 24,97 °C
Umidade média: 57,69%
PM2.5 máximo: 42,58 µg/m³
PM10 máximo: 74,31 µg/m³
As 24 medições representam duas leituras simultâneas a cada cinco minutos, realizadas pelos dois sensores da cidade ao longo de uma hora.
Evidências a inserir: [PRINT 1 — seis sensores]; [PRINT 2 — 1.728 medições]; [PRINT 3 — consulta S001]; [PRINT 4 — execução da agregação]; [PRINT 5 — HGETALL no Redis].

6. Testes adicionais — Etapa II

A Etapa II solicita scripts de teste com dados diferentes dos já armazenados. [PENDENTE: descrever os novos dados utilizados, o comando de execução e o resultado observado após a realização do teste. Não registrar como concluído antes da validação.]

7. Organização e compartilhamento no GitHub

O repositório deverá conter README.md com os nomes dos integrantes, uma pasta codigo/ para os quatro arquivos Python e uma pasta documentacao/ para este relatório técnico. Também deverá ser compartilhado com a professora por meio do usuário MonicaTI. [PENDENTE: inserir URL do repositório e confirmar compartilhamento.]

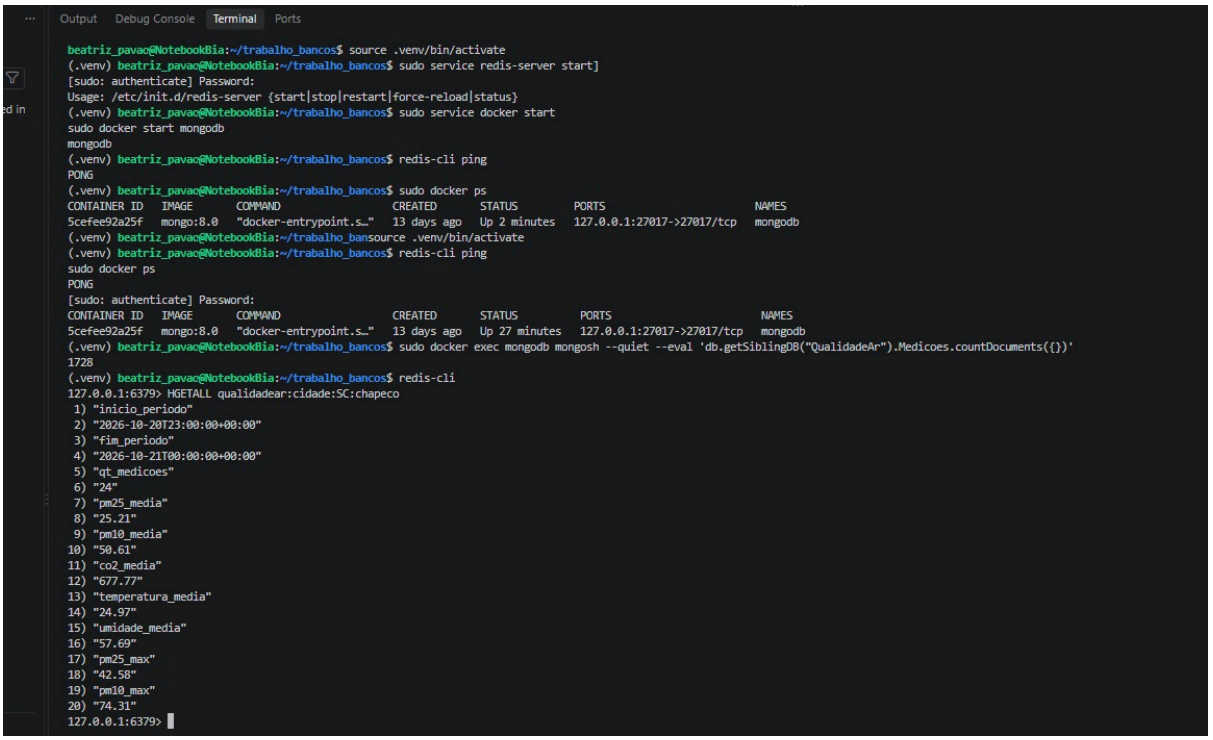
8. Dificuldades e soluções

Na configuração inicial, o VS Code indicou um interpretador Python inválido. A execução pelo Ubuntu (WSL), com o ambiente virtual .venv ativado, permitiu prosseguir. Os servidores foram iniciados e testados com redis-cli ping e docker ps. Na primeira consulta, o identificador foi digitado junto com o nome da cidade, produzindo “Sensor nao encontrado”; repetindo a execução com apenas S001, a consulta retornou os registros esperados.

9. Conclusão

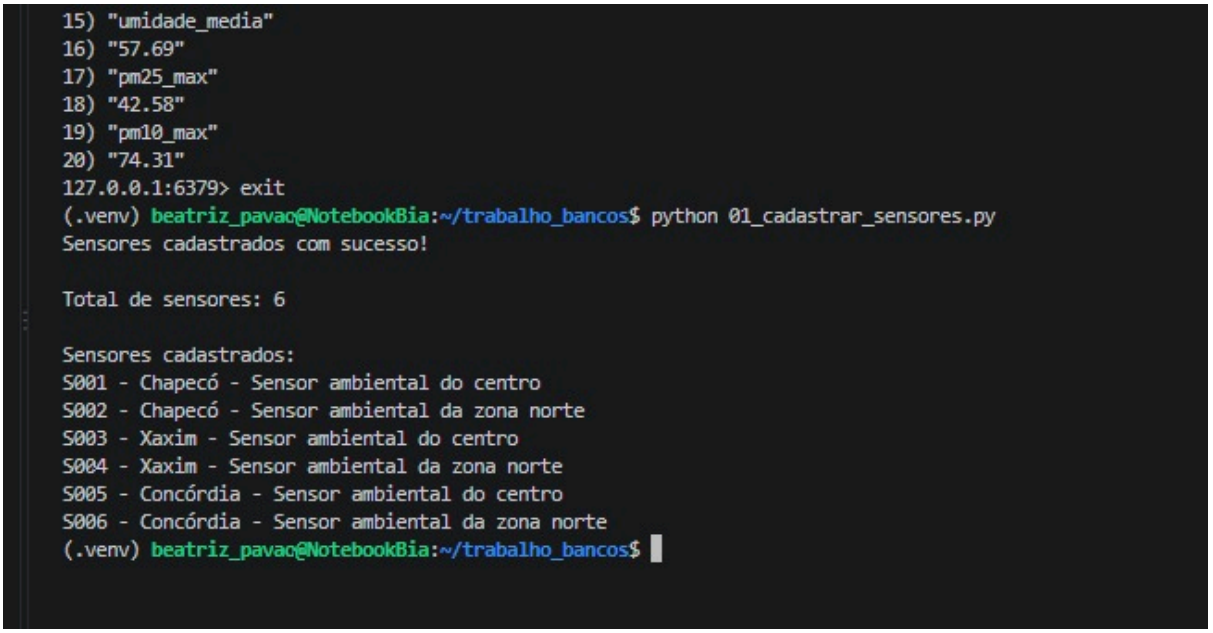
O protótipo demonstrou que o MongoDB pode armazenar sensores e histórico de medições, enquanto o Python consulta e agrega os dados e o Redis guarda os resumos horários em Hashes para leitura rápida. A implementação principal da Etapa I foi validada no terminal. A entrega da Etapa II depende da inclusão dos testes adicionais, do diagrama final, da documentação e do repositório compartilhado.

Testes:



```
Output  Debug Console  Terminal  Ports

beatriz_pavao@NotebookBia:~/trabalho_bancos$ source .venv/bin/activate
(.venv) beatriz_pavao@NotebookBia:~/trabalho_bancos$ sudo service redis-server start]
[sudo: authenticate] Password:
Usage: /etc/init.d/redis-server {start|stop|restart|force-reload|status}
(.venv) beatriz_pavao@NotebookBia:~/trabalho_bancos$ sudo service docker start
sudo docker start mongodb
mongodb
(.venv) beatriz_pavao@NotebookBia:~/trabalho_bancos$ redis-cli ping
PONG
(.venv) beatriz_pavao@NotebookBia:~/trabalho_bancos$ sudo docker ps
CONTAINER ID   IMAGE       COMMAND                  CREATED       STATUS         PORTS                      NAMES
5cefee92a25f   mongo:8.0   "docker-entrypoint.s…"   13 days ago   Up 2 minutes   127.0.0.1:27017->27017/tcp   mongodb
(.venv) beatriz_pavao@NotebookBia:~/trabalho_bansource .venv/bin/activate
(.venv) beatriz_pavao@NotebookBia:~/trabalho_bancos$ redis-cli ping
sudo docker ps
PONG
[sudo: authenticate] Password:
CONTAINER ID   IMAGE       COMMAND                  CREATED       STATUS         PORTS                      NAMES
5cefee92a25f   mongo:8.0   "docker-entrypoint.s…"   13 days ago   Up 27 minutes   127.0.0.1:27017->27017/tcp   mongodb
(.venv) beatriz_pavao@NotebookBia:~/trabalho_bancos$ sudo docker exec mongodb mongosh --quiet --eval 'db.getSiblingDB("QualidadeAr").Medicoes.countDocuments({})'
1728
(.venv) beatriz_pavao@NotebookBia:~/trabalho_bancos$ redis-cli
127.0.0.1:6379> HGETALL qualidadear:cidade:SC:chapeco
 1) "inicio_periodo"
 2) "2026-10-20T23:00:00+00:00"
 3) "fim_periodo"
 4) "2026-10-21T00:00:00+00:00"
 5) "qt_medicoes"
 6) "24"
 7) "pm25_media"
 8) "25.21"
 9) "pm10_media"
10) "50.61"
11) "co2_media"
12) "677.77"
13) "temperatura_media"
14) "24.97"
15) "umidade_media"
16) "57.69"
17) "pm25_max"
18) "42.58"
19) "pm10_max"
20) "74.31"
127.0.0.1:6379>
```

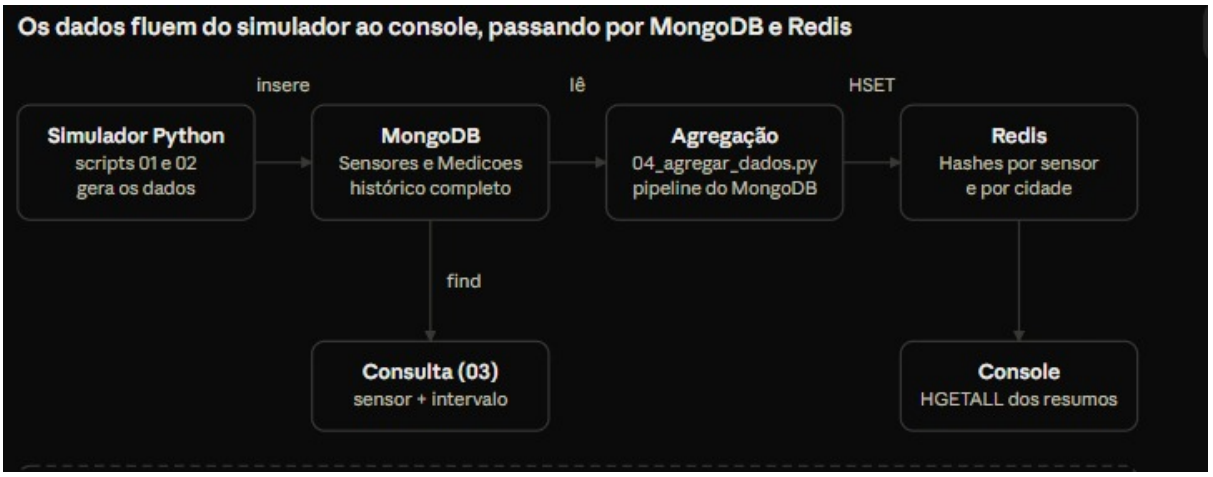


```
15) "umidade_media"
16) "57.69"
17) "pm25_max"
18) "42.58"
19) "pm10_max"
20) "74.31"
127.0.0.1:6379> exit
(.venv) beatriz_pavao@NotebookBia:~/trabalho_bancos$ python 01_cadastrar_sensores.py
Sensores cadastrados com sucesso!

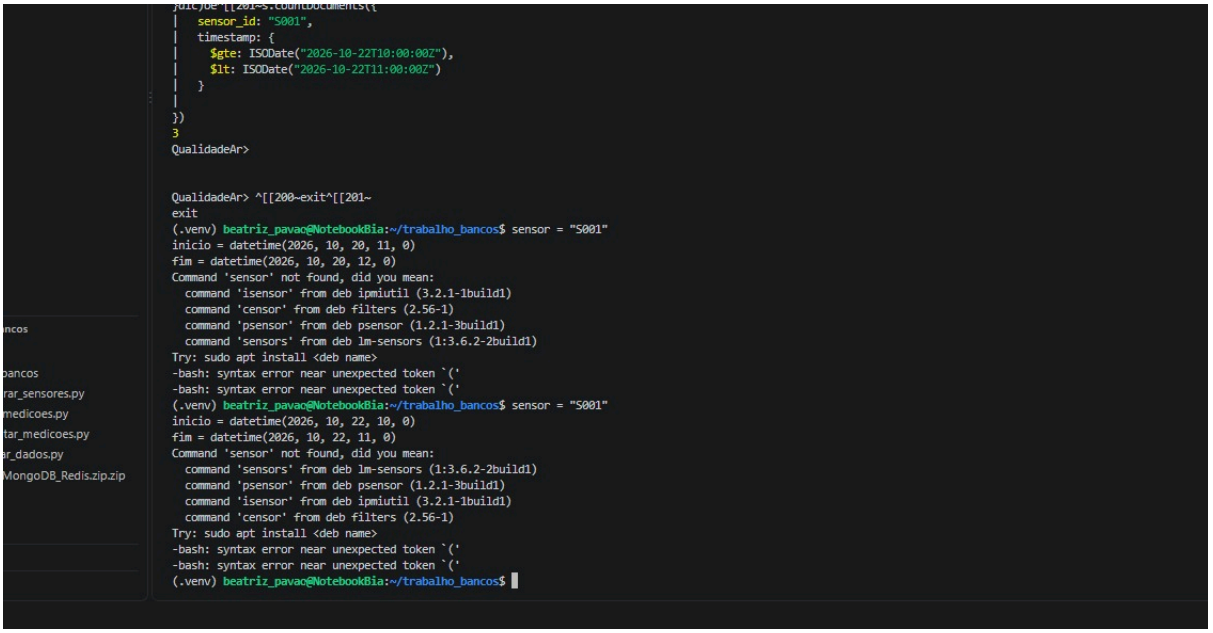
Total de sensores: 6

Sensores cadastrados:
S001 - Chapecó - Sensor ambiental do centro
S002 - Chapecó - Sensor ambiental da zona norte
S003 - Xaxim - Sensor ambiental do centro
S004 - Xaxim - Sensor ambiental da zona norte
S005 - Concórdia - Sensor ambiental do centro
S006 - Concórdia - Sensor ambiental da zona norte
(.venv) beatriz_pavao@NotebookBia:~/trabalho_bancos$
```

Arquitetura:



Testes:



10. Referências

MONGODB. MongoDB Documentation. https://www.mongodb.com/docs/
REDIS. Redis Documentation. https://redis.io/docs/
PYMONGO. PyMongo Documentation. https://pymongo.readthedocs.io/
OPENAI. ChatGPT. Ferramenta de IA utilizada como apoio ao desenvolvimento e à redação do relatório, outubro de 2026.
PEREIRA, Mônica Tissiani De Toni. Estudo de Caso: Monitoramento da Qualidade do Ar — Integração MongoDB + Redis com Python. Etapas I e II. Unochapecó, 2026.